7. Manual del programador — Compilar y usar las librerías

Este manual explica, paso a paso, **cómo compilar** y **cómo usar** las dos

librerías cliente ADS (`LibPhithon` en Python y `LibC#` en C#). Está pensado para

que un programador que vaya a integrarlas sepa exactamente **qué método llamar y

con qué parámetros**.

- **LibPhithon** → Python puro (solo biblioteca estándar).
- **LibC#** → .NET 6 (solo la BCL, sin paquetes NuGet).

7.1. Compilación / instalación

7.1.1. Python — LibPhithon

No requiere compilar: es Python puro. Solo hay que instalarla en el entorno

(editable) o añadir la carpeta al `PYTHONPATH`.

```
cd LibPhithon
pip install -e .
```

Requisito: **Python ≥ 3.8**. No usa paquetes de terceros.

Comprobar que funciona (los tests):

```
cd LibPhithon
python -m unittest discover -s tests -v
```

7.1.2. C# — LibC#

Compila la librería con el SDK de .NET 6:

```
cd LibC#
dotnet build src/Beckhoff.Ads/Beckhoff.Ads.csproj
```

El resultado es la DLL `Beckhoff.Ads.dll` dentro de `src/Beckhoff.Ads/bin/Debug/net6.0/`.

Para **usarla desde tu propio proyecto**, añade una referencia al proyecto (o a la

DLL):

```
<ProjectReference Include="..\LibC#\src\Beckhoff.Ads\Beckhoff.Ads.csproj" />
```

Requisito: **.NET 6 SDK** o superior.

Comprobar que funciona (los tests, un ejecutable auto-contenido):

```
cd LibC#
dotnet run --project tests/Ads.Tests/Ads.Tests.csproj
```

7.2. Uso rápido (lo esencial en 1 minuto)

En los dos casos el flujo es el mismo:

1. Crear el cliente con la **dirección AMS del PLC** (NetId + puerto AMS) y la

dirección TCP (IP + puerto).

2. **Conectar**.
3. **Leer/Escribir** variables por su nombre (o por índice/offset).
4. **Cerrar** la conexión.

7.2.1. Python

```
from ads import AdsClient

with AdsClient(ams_net_id="127.0.0.1.1.1", ams_port=851,
               ip_address="127.0.0.1", tcp_port=48898) as c:
    # Estado del PLC
    ads_state, device_state = c.read_state()

    # Leer una variable por nombre (devolviendo el valor Python descodificado)
    contador = c.read_by_name("MAIN.nCounter", "UINT")

    # Escribir una variable por nombre
    c.write_by_name("MAIN.bStart", True, "BOOL")
```

7.2.2. C#

```
using Beckhoff.Ads;

using var client = new TcAdsClient(
    amsNetId: "127.0.0.1.1.1",
    amsPort: AdsPorts.PlcTc3,       // 851
    ipAddress: "127.0.0.1",
    tcpPort: AdsPorts.TcpPort);     // 48898

client.Connect();

var (state, deviceState) = client.ReadState();
ushort counter = client.ReadValue<ushort>("MAIN.nCounter", "UINT");
client.WriteValue<bool>("MAIN.bStart", true, "BOOL");
```

7.3. Referencia de métodos (qué llamar y con qué parámetros)

7.3.1. Python — AdsClient

Constructor:

```
AdsClient(ams_net_id="127.0.0.1.1.1", ams_port=851,
          ip_address="127.0.0.1", tcp_port=48898,
          timeout=5.0, local_port=0)
```

Parámetro	Tipo	Qué es
`ams_net_id`	`str`	NetId AMS del destino (ej. `"127.0.0.1.1.1"`).
`ams_port`	`int`	Puerto AMS del destino (851 = PLC Runtime).
`ip_address`	`str`	Host/IP donde conecta el socket TCP.
`tcp_port`	`int`	Puerto TCP de AMS/TCP (48898).
`timeout`	`float`	Timeout por transacción (segundos).
`local_port`	`int`	Puerto AMS local (0 = aleatorio).

Métodos principales:

Método	Parámetros	Devuelve
`open()` / `close()`	—	Conexión TCP abierta/cerrada.
`read_state()`	—	`(ads_state, device_state)`.
`read_device_info()`	—	`(major, minor, build, name)`.
`read_by_name(name, plc_datatype=None)`	`name`: nombre del símbolo; `plc_datatype`: tipo PLC opcional	Valor Python (si se da tipo) o `bytes`.
`write_by_name(name, value, plc_datatype=None)`	`name`, `value` (bytes o valor Python), `plc_datatype`	—
`read_by_handle(handle, size)`	handle + tamaño	`bytes`.
`write_by_handle(handle, data)`	handle + bytes	—
`get_handle(name)` / `release_handle(handle)`	nombre / handle	handle / —
`read_symbol_info(name)`	nombre	`SymbolInfo` (con `.size`, `.name`, `.type_name`…).
`add_device_notification(ig, io, cb_length, trans_mode=4, max_delay=0, cycle_time=1000000, callback=None)`	índice/offset, longitud, modo, tiempos, callback	handle de notificación.
`del_device_notification(handle)`	handle	—
`sum_read(entries)`	lista de `(ig, io, length)`	lista de `bytes`.
`sum_write(entries)`	lista de `(ig, io, data)`	—

Tipos de datos (módulo `ads.datatypes`):

Función	Parámetros	Devuelve
`size_of(plc_type)`	tipo PLC (`"UINT"`, `"REAL"`…)	tamaño en bytes.
`encode_value(value, plc_type)`	valor Python + tipo	`bytes`.
`decode_value(data, plc_type)`	bytes + tipo	valor Python.

7.3.2. C# — TcAdsClient

Constructor:

```
new TcAdsClient(amsNetId: "127.0.0.1.1.1", amsPort: 851,
                ipAddress: "127.0.0.1", tcpPort: 48898,
                timeoutMs: 5000)
```

Parámetro	Tipo	Qué es
`amsNetId`	`string`	NetId AMS del destino.
`amsPort`	`ushort`	Puerto AMS del destino (851).
`ipAddress`	`string`	Host/IP TCP.
`tcpPort`	`int`	Puerto TCP (48898).
`timeoutMs`	`int`	Timeout por transacción (ms).

Métodos principales:

Método	Parámetros	Devuelve
`Connect()` / `Disconnect()`	—	Conexión abierta/cerrada.
`ReadState()`	—	`(AdsState State, ushort DeviceState)`.
`ReadDeviceInfo()`	—	`AdsDeviceInfo`.
`ReadValue<T>(name, plcType=null)`	nombre + tipo PLC	valor `T` descodificado.
`WriteValue<T>(name, value, plcType=null)`	nombre + valor + tipo	—
`ReadSymbol(name, size)` / `ReadSymbol(name)` / `ReadSymbol(name, plcType)`	nombre (y tamaño o tipo)	`byte[]`.
`WriteSymbol(name, value)`	nombre + `byte[]`	—
`CreateVariableHandle(name)` / `DeleteVariableHandle(handle)`	nombre / handle	`uint` / —
`ReadByHandle(handle, length)` / `WriteByHandle(handle, data)`	handle + longitud/bytes	`byte[]` / —
`GetSymbolInfo(name)`	nombre	`AdsSymbolInfo` (`.Size`, `.Name`…).

`AddNotificationByName(name, plcType, transMode, maxDelay, cycleTime, callback)`	nombre, tipo, modo, tiempos, callback	`uint` (handle).
`DeleteDeviceNotification(handle)`	handle	—
`SumRead(entries)` / `SumWrite(entries)`	lista de tuplas `(ig, io, length)` / `(ig, io, data)`	lista de `byte[]` / —

7.4. Tipos PLC admitidos

`BOOL`, `BYTE`, `SINT`, `USINT`, `INT`, `UINT`, `WORD`, `DINT`, `UDINT`, `DWORD`,

`REAL`, `TIME`, `TOD`, `DT`, `DATE`, `LINT`, `ULINT`, `LREAL`, `STRING(n)`,

`WSTRING(n)` y `ARRAY[a..b] OF T`.

Todos los valores se codifican/decodifican en **little-endian** (orden de bytes de los PLC Beckhoff).

7.5. Ejemplos completos

7.5.1. Python — leer y escribir con tipos

```
from ads import AdsClient

with AdsClient(ams_net_id="127.0.0.1.1.1", ams_port=851,
               ip_address="127.0.0.1", tcp_port=48898) as c:
    # Estado
    estado, disp = c.read_state()
    print(f"Estado: {estado}, dispositivo: {disp}")

    # Leer con tipo (devuelve valor Python)
    nivel = c.read_by_name("MAIN.iLevel", "INT")
    temp = c.read_by_name("MAIN.fTemp", "REAL")
    print(f"Nivel={nivel}, Temperatura={temp}")

    # Escribir con tipo
    c.write_by_name("MAIN.bStart", True, "BOOL")
    c.write_by_name("MAIN.nCounter", 42, "UINT")
```

7.5.2. C# — leer y escribir con tipos

```
using Beckhoff.Ads;

using var client = new TcAdsClient(
    amsNetId: "127.0.0.1.1.1",
    amsPort: AdsPorts.PlcTc3,
    ipAddress: "127.0.0.1",
```

```
    tcpPort: AdsPorts.TcpPort);

client.Connect();

var (estado, disp) = client.ReadState();
short nivel = client.ReadValue<short>("MAIN.iLevel", "INT");
float temp = client.ReadValue<float>("MAIN.fTemp", "REAL");
Console.WriteLine($"Estado={estado}, Nivel={nivel}, Temp={temp}");

client.WriteValue<bool>("MAIN.bStart", true, "BOOL");
client.WriteValue<ushort>("MAIN.nCounter", 42, "UINT");
```

7.5.3. Notificaciones (cuando cambia una variable)

Python:

```
def al_cambiar(sample):
    print(f"Notificación handle={sample.handle}: {sample.data.hex()}")

c = AdsClient(ams_net_id="127.0.0.1.1.1", ams_port=851,
              ip_address="127.0.0.1", tcp_port=48898)
c.open()
handle = c.add_notification_by_name("MAIN.nCounter", "UINT",
                                    trans_mode=4, cycle_time=1000000,
                                    callback=al_cambiar)
# ... la notificación llega en segundo plano ...
c.del_device_notification(handle)
c.close()
```

C#:

```
uint handle = client.AddNotificationByName(
    "MAIN.nCounter", "UINT",
    AdsTransMode.ServerOnChange, 0, 1000000,
    sample => Console.WriteLine($"Notificación: {sample.Data.Length} bytes"));
// ...
client.DeleteDeviceNotification(handle);
```

7.5.4. Leer/escribir estructuras completas

Python:

```
from ads import AdsClient, StructMember, StructDefinition

posicion = StructDefinition([
    StructMember("X", "LREAL", 0),
    StructMember("Y", "LREAL", 8),
    StructMember("Activo", "BOOL", 16),
], total_size=24)

with AdsClient(...) as c:
    valores = c.read_struct("GVL.Posicion", posicion)   # dict {miembro: valor}
    c.write_struct("GVL.Posicion", posicion, {"X": 3.0, "Y": 4.0, "Activo":
False})
```

C#:

```
var posicion = new PlcStruct(new[]
{
    new PlcMember("X", "LREAL", 0),
    new PlcMember("Y", "LREAL", 8),
    new PlcMember("Activo", "BOOL", 16),
}, totalSize: 24);

var valores = client.ReadStruct("GVL.Posicion", posicion);   // Dictionary<string,
object?>
client.WriteStruct("GVL.Posicion", posicion, new Dictionary<string, object?>
    { ["X"] = 3.0, ["Y"] = 4.0, ["Activo"] = false });
```

7.6. Resumen: qué método usar según la necesidad

Quiero…	**Python**	**C#**
Conectar	`AdsClient(...).open()`	`new TcAdsClient(...).Connect()`
Leer estado	`read_state()`	`ReadState()`
Leer una variable por nombre	`read_by_name("X", "TIPO")`	`ReadValue<T>("X", "TIPO")`
Escribir una variable por nombre	`write_by_name("X", valor, "TIPO")`	`WriteValue<T>("X", valor, "TIPO")`
Leer bytes crudos	`read_by_name("X")`	`ReadSymbol("X")`
Leer una estructura completa	`read_struct("X", def)`	`ReadStruct("X", def)`
Escribir una estructura completa	`write_struct("X", def, vals)`	`WriteStruct("X", def, vals)`
Leer por índice/offset	`read_index(ig, io, size)`	`ReadAny(ig, io, length)`
Suscribir notificación	`add_device_notification(...)`	`AddDeviceNotification(...)`
Cerrar	`close()`	`Disconnect()`